Unit 4 — Enterprise Solutioning & Contract-Driven Architecture (Solu-2)

University AI Compute Fabric — From System Architecture to a Sprint-Ready Backlog

Course Module: Unit 4 — Contracts, Internal Design, Data Modeling, and Architecture Implementation
Target Outcome (RAA4): Derive code interfaces, technical criteria, and data schemas from a plan to subsequently implement software components consistent with the system's functional intent.
Delivery: Pairs (the same pairs and the same module / system selected for Plan-2 assignment), AI Governance Level 3
Prerequisites: Plan-2 package for your module available (prd.md, DESIGN.md, EXPERIENCE.md, ARCHITECTURE.md with AD-1..n).
Deliverable: Detailed Architecture Package + Test Preparation Package + epics.md + sprint-status.yaml with a certified readiness gate.

The Solutioning Stop-Point Rule (same as Solu-1): Solutioning halts at Epic/Story creation and the Sprint Readiness Audit (bmad-sprint-planning). Autonomous implementation agents (bmad-build, bmad-build-auto) are NOT invoked in this unit. You may write interface stubs and failing tests only but no production logic.

1. Overview

In **Solu-1** you worked individually on a given system (Smart Solar Flow) and the contracts, schemas, and tests were largely framed for you. In **Solu-2** the roles reverse:

	Solu-1 (Smart Solar Flow)	Solu-2 (University AI Compute Fabric)
Team	Individual	**Pair** (Architect-lead + Test-lead, roles swap in Phase 3)
Starting point	Provided PRD + ARCHITECTURE-SPINE.md (AD-16..23)	**Your own** Plan-2 package and AD-n invariants
What you choose	Nothing: functions were given	**You choose at least** the 3 core operations of your module (guided by the Seed Table, Annex A)
Stack	TypeScript / PostgreSQL / MQTT	Free within the Plan-2 baseline: Python (FastAPI/Pydantic/pytest) **or** TypeScript (Node/Vitest); PostgreSQL **or** SQLite; all GPU/telemetry data simulated
Output	Contracts, DDL, DTOs, ATDD, stories	Full detailed design, risk-based test preparation, **3–4 epics / 9–16 stories in epics.md**, sprint-status.yaml

Your call: You will lead the autonomous AI engineering council (**Winston** – Architect, **Amelia** – Dev, **Murat** – Test Architect, **John** – PM, plus **Freya/Sally** for UX consistency checks) to turn your high-level module architecture into **inviolable, machine-enforceable contracts** and a backlog that a coding agent could implement without inventing a single decision.

SOLU-2 WORKSHOP OUTLINE

PHASE 0: Intake & Architecture Gap Audit:

- /bmad-review (adversarial) on the 3 core operations, list gaps in ARCHITECTURE.md
- ARCHITECTURE.md: Decide stack (Python/TS, Postgres/SQLite)

PHASE 1: Formal Behavioral Contracts (P, Q, I):

- /bmad-architecture: 3 operations x (P, Q, I) + typed error catalog
- /bmad-agent-architect (Winston): Every invariant bound to an AD-n

PHASE 2: Ports & Adapters, Schemas, DTOs, Events:

- /bmad-architecture: Hexagonal layer map, DDL migration, DTO/OpenAPI
- /bmad-agent-dev (Amelia, audit): Simulated telemetry / clock / GPU adapters

PHASE 3: Test Preparation (TEA):

- /bmad-testarch-test-design, -framework - Risk matrix P0-P3, red-phase ATDD suite, CI gate,
- -atdd, -ci, -nfr, -trace, -test-review NFR evidence plan, traceability matrix

PHASE 4: epics.md + Sprint Planning:

- /bmad-create-epics-and-stories: 3-4 vertical epics, 3-5 contract-bound stories each
- /bmad-sprint-planning: Readiness gate PASS -> sprint-status.yaml

PHASE 5: Wrap-up: Decision log, triage, submission

2. Source Materials & Required Inputs

Before starting, make sure the following exist in your repository. If one is missing or contradictory, that is your **first finding** (Phase 0).

#	Input	Source	Used in
1	project - university_ai_compute_ management_platform_brief.pdf	Course project brief	Phase 0 (institutional context, tensions)
2	prd.md (FRs, NFRs, non-goals)	Plan-2 Phase 1	Phases 1, 3, 4
3	DESIGN.md, EXPERIENCE.md, scenario files (4 scenarios)	Plan-2 Phase 2	Phases 2, 4 (every UI state needs a backend contract)
4	ARCHITECTURE.md with **6–8 AD-n invariants**, component + sequence diagrams, data model sketch	Plan-2 Phase 3	All phases
5	Plan-2 review reports (review-*.md) with triage	Plan-2	Phase 0 (unresolved Defer items)
6	**Unit 4 slides** — *Contract-Driven Architecture & Enterprise Solutioning*	Course	Reference for P/Q/I, Ports & Adapters, TEA
7	**Solu-1 solution** (task-solu1_SmartSolarFlow_Workshop)	Course	Worked pattern: contracts → DDL/DTO → ATDD → Story → gate

Hardware agnosticism still applies. Physical NVIDIA GPUs are never required for development. Every adapter that touches GPUs, Prometheus/DCGM metrics, Kubernetes, vLLM, or the system clock must be an outbound port with a simulated driven adapter (fixtures, fake clock, scripted telemetry).

3. Phase 0 — Intake & Architecture Gap Audit

Goal: Convert Plan-2's ARCHITECTURE.md from "approved" to "buildable" by finding what it does not yet decide.

Step 0.1 — Select the 3+ Core Operations

From your PRD's FRs and your 4 mandatory scenarios, pick **3+ operations** that carry the **non-trivial decision logic** (algorithm, policy, state machine) of your module. They must together cover:

- at least **one** mutation that can be raced, repeated, or interrupted (idempotency / concurrency risk);
- at least **one** automated decision that must return an explainable, cited justification (Plan-2 constraint 4);
- the **Mandatory Edge Case** of your module (see Annex A).

Use the Seed Table (Annex A) as a starting suggestion only. You may substitute operations if you justify the choice in the decision log.

Step 0.2 — Architecture Gap Audit

Run an adversarial review limited to *implementability*:

/bmad-review _bmad-output/planning/ARCHITECTURE-SPINE.md --lens adversarial (check your specific architecture file path)

Then complete this table (one row per gap with at least **6 gaps**):

#	Gap in **ARCHITECTURE-SPINE.md** (what is decided by "nothing")	Example: where an AI/dev agent would invent a decision	Resolved in Phase	Owner (A/B)
1				

Typical gap classes: undefined error codes, missing idempotency keys, unspecified time zone/clock source, unbounded queues, no ID format, missing NOT NULL/CHECK, an AD-n with no code-level binding, a UI state with no backend contract.

Step 0.3 — Stack Declaration (5 lines)

Record in ARCHITECTURE.md "Stack Decisions": language, web framework, persistence engine, test framework, and **how time is injected** (a Clock port is mandatory for any module with time rules).

Phase 0 output: reviews/review-arch-implementability.md (triaged: Accept / Defer / Reject), updated stack section.

4. Phase 1 — Formal Behavioral Contracts (P, Q, I)

Goal: Write Design-by-Contract specifications for your 3+ core operations, with **zero elastic language**.

Banned words in a contract**:** *efficiently, appropriately, securely, quickly, gracefully, reasonable, robust, properly, as needed, scalable, user-friendly, "etc."* Every such word must be replaced by a number, a type, an enumeration, a status code, or a state name.

Contract Template (use as a template per operation)

```
### Operation: `<name>(<typed parameters>)`
- **Layer:** Inbound Port | Core Domain | Outbound Port   (Phase 2 will confirm)
- **Precondition (P):**
  - P1: <parameter type, range, nullability>  -> violation: <TYPED_ERROR, HTTP status>
  - P2: <state/authorization/entitlement requirement> -> violation: <TYPED_ERROR, HTTP status>
- **Postcondition (Q):**
  - Q1: <return value with exact schema>
  - Q2: <state transition / DB mutation, stated as a predicate>
  - Q3: <audit event / metric emitted, exact name>
  - Q4: <explainability payload: rule IDs / values cited>
  - Q-fail: <what is guaranteed unchanged when a precondition fails>
- **Invariant (I):**
  - I1: <statement that holds before AND after every call>   Bound to: AD-<n>
- **Concurrency / idempotency:** <what happens on 2 simultaneous calls; repeated call with same key>
- **Time:** <which Clock port; boundary semantics, e.g. half-open interval [start, end)>
- **Latency budget (NFR):** <p95 < X ms at N concurrent requests, from PRD NFR-n>
```

Step 1.1 — Complete the Behavioral Contract Matrix

Produce planning/detailed-design/contracts.md containing, for each of your 3+ operations:

1. The filled template above.
2. A **Typed Error Catalog** (shared by all operations): CODE | HTTP/exit status | trigger | human-readable explanation template | retryable (Y/N).
3. An **Invariant Binding Table**:
4.

Invariant	**AD-n** it enforces	Enforcement point (function / DB constraint / state-machine guard)	Test that turns RED if removed

If an invariant has no enforcement point *other than "the developer must remember"*, it is not an invariant — go back and add a DB CHECK/UNIQUE/exclusion constraint, a state-machine guard, or a pure-function property.

5. Phase 2 — Internal Interfaces, Schemas & Contracts

Goal: Place every component in a Hexagonal layer, then author concrete DTOs/events and a migration that **physically enforce** the contracts.

Step 2.1 — Ports & Adapters Architecture Mapping

Produce planning/detailed-design/ports-and-adapters.md. Every component of your module must appear exactly once:

Component (file path)	Hexagonal layer	Depends on (allowed)	Governed by (**AD-n** / contract)
api/routes/...	**Driving Adapter** (HTTP/CLI)	Inbound Port only	
ports/in/...	**Inbound Port** (use-case interface)	DTOs only	
domain/...	**Core Domain & Invariants** (pure)	nothing outside domain/	
ports/out/...	**Outbound Port** (repository, clock, telemetry, notifier)	domain types	
adapters/db/...	**Driven Adapter** (persistence)	Outbound Port	
adapters/sim/...	**Driven Adapter** (simulated GPU/Prometheus/vLLM/K8s, fake clock)	Outbound Port	

Hard rules (the Architecture Review will check them): 1. domain/ must not import the web framework (FastAPI/Express), the ORM/DB driver, datetime.now()/Date.now(), random, or any network client. 2. Time and randomness enter only through the Clock / IdGenerator ports. 3. Every GPU/telemetry/cluster interaction is behind an outbound port with a **simulated** adapter. 4. DTOs are primitive-typed; no framework object in a DTO or port signature.

Draw the **component diagram** in Mermaid (update the Plan-2 diagram so each box carries its layer tag).

Step 2.2 — Concrete Schema, DTO & Event Authoring

Produce, under planning/detailed-design/schemas/:

Artifact	Minimum content
001_<module>_schema.sql	Every entity in your Plan-2 data model with PK/FK, NOT NULL, CHECK, UNIQUE, and **at least 2 constraints that enforce an invariant physically** (e.g., CHECK no-overdraw, partial unique index, exclusion constraint). Include created_at/updated_at and an audit_events table.
dto.py or dto.ts	One request DTO + one response DTO per operation, one ErrorBody DTO, one ExplanationDTO (rule IDs, observed values, thresholds).

Artifact	Minimum content
events.md / JSON Schema	Every event/telemetry payload your module emits or ingests, with eventId, occurredAt (UTC, ISO-8601), schemaVersion.
openapi.yaml (headless modules: mandatory; others: for all ingress routes)	Paths, request/response schemas, **all** error responses from the Typed Error Catalog.

Atomicity beats cleverness. If two actors can race, the schema must serialize them: a single conditional UPDATE ... WHERE <predicate> RETURNING *, a partial unique index, a Postgres exclusion constraint, or BEGIN IMMEDIATE in SQLite. *"SELECT then UPDATE"* is a defect.

Step 2.3 — Contract ↔ Schema Cross-Check (5 min)

Re-open your Binding Table from Phase 1. For **every** I clause, point to the exact CHECK/UNIQUE/guard line in the schema or domain file. Any clause with no pointer is a defect: fix the schema (preferred) or soften the contract with a documented trade-off (an AD-n Trade-off: entry).

Phase 1–2 gate: run

```
/bmad-review planning/detailed-design/ --lens adversarial
/bmad-review planning/detailed-design/ --lens edge-case-hunter
```

and save the triaged output to reviews/review-detailed-design.md.

---

6. Phase 3 — Test Preparation with the BMad TEA Framework (45 min)

Goal: Before any production code exists, prepare the entire test strategy: risk matrix, framework, **executable red-phase ATDD suite**, CI quality gate, NFR evidence plan, and a traceability matrix.

Pair roles for this phase: the partner who led Phase 1–2 becomes the **Reviewer**; the other partner becomes the **Test Lead** and drives Murat.

Step 3.1 — Risk-Based Test Design

```
/bmad-testarch-test-design      # system-level mode, then epic-level mode after Phase 4
```

Produce test/test-design.md containing:

1. **Risk register** (≥ 8 risks): ID | risk | probability (1–3) | impact (1–3) | score | priority P0–P3 | mitigating contract/AD-n. Every module's *Mandatory Edge Case* must be **P0**.
2. **Test level allocation:** which behaviour is verified as *unit* (pure domain), *contract/integration* (adapter + DB), *API* (HTTP), or *E2E* (UI scenario). Pure domain logic must have the highest share.
3. **Coverage thresholds:** e.g., 100% of P0 contracts have ≥ 1 red test; line coverage target for domain/ ≥ 90%.

4. **Testability gaps** in the architecture (non-injectable clock, hidden global state, no seed fixtures) → feed back into Phase 0 gaps.

Step 3.2 — Framework Initialization (scaffold only)

/bmad-testarch-framework

Create the harness (pytest + pytest-asyncio + hypothesis, **or** vitest + fast-check; Playwright only if your prototype has a UI) with these **mandatory fixtures**:

Fixture	Purpose
fake_clock	Deterministic, advanceable time (no real sleep)
seed_fleet	The Plan-2 fleet profile: 31 × (1 × 24 GB) + 1 × (2 × 48 GB), all HEALTHY
sim_telemetry	Scripted GPU/heartbeat/metric injector (fault generator)
db	Ephemeral Postgres (Docker Compose) or in-memory SQLite, migrated from your 001_*.sql
explain_assert	Helper that asserts a decision carries ≥ 1 rule ID and the observed values

Step 3.3 — Write the ATDD Red-Phase Suite

/bmad-testarch-atdd

Create test/atdd/<module>_contracts.test.(py|ts) with **at least 2 tests per operation per category**:

Category	Minimum
Happy path (Q holds)	1 per operation
Precondition violation (P) → typed error, **no state mutation**	1 per operation
Invariant (I) under concurrency/repetition/boundary	1 per operation
Mandatory Edge Case of your module	≥ 1, assertions on final state
Explainability (decision has cited rule IDs)	≥ 1

A red test must fail for the right reason. It must call your interface stub (raising NotImplementedError) and contain real assertions on *return value, resulting* state*, emitted events, and — for rejections — what did NOT happen* (zero mutations, zero side-effect calls). assert True, empty it(...) bodies, and assertions on mocks you configured yourself are tautologies and will be graded as defects.

Step 3.4 — NFR Evidence Plan

```
/bmad-testarch-nfr
```

Produce test/nfr-evidence.md: one row per PRD NFR — NFR | metric & threshold | how measured (tool, fixture, N) | evidence artifact | status (PLANNED/EVIDENCED). At minimum: one **performance** NFR (with a concurrency number), one **reliability/failure** NFR (timeout, retry, or recovery time), and one **security/isolation** NFR (e.g., token never logged, role isolation, destructive action needs confirmation).

Step 3.6 — Test Quality Review & Traceability

```
/bmad-testarch-test-review test/atdd/
/bmad-testarch-trace          # run again after Phase 4 when stories exist
```

Produce test/traceability-matrix.md:

PRD FR/NFR	Scenario (UX)	**AD-n**	Contract clause	Test ID	Story ID	Status

Phase 3 gate: no P0 risk without a red test; no AD-n without a test that goes red if the rule is violated.

Step 3.7 — TEA Skill Matrix (complete in your own words)

Quality / Test Workflow	BMad TEA Skill	Artifact **you** produced	Defect it caught in your package
Risk-Based Test Strategy	/bmad-testarch-test-design		
Framework Initialization	/bmad-testarch-framework		
ATDD Test Generation	/bmad-testarch-atdd		
CI Quality Gate Scaffold	/bmad-testarch-ci		
NFR Evidence Audit	/bmad-testarch-nfr		
Test Quality Review	/bmad-testarch-test-review		
Traceability Matrix & Gate	/bmad-testarch-trace		

7. Phase 4 — epics.md and Sprint Planning (40 min)

Goal: Produce a **vertical-slice backlog** whose every story is bound to contracts, schemas, AD-n invariants and ATDD tests, then pass the readiness gate and generate sprint-status.yaml.

Step 4.1 — Epic & Story Creation

```
/bmad-create-epics-and-stories    # John (PM) + Amelia (Dev) review for implementability
```

Write planning/epics.md following these rules.

Structure rules * **3–4 epics**, each a *vertical slice* that delivers user-visible value for a named protagonist (not "Database layer", not "API layer"). * **3–5 stories per epic** (9–16 total). Each story is independently completable; **no forward dependencies** (a story may only depend on earlier stories). * **Epic 1 is a walking skeleton:** seed fixtures + one end-to-end happy path through all layers + the simulated adapters. * The **Mandatory Edge Case** and **≥ 2 failure states** of your module each have a dedicated story.

Story template (copy for every story)

Step 4.2 — Sprint Readiness Audit

```
/bmad-sprint-planning
```

The skill (a) judges whether a developer could implement your epics **without inventing decisions nothing records**, then (b) on a passing gate parse epics.md and generates sprint-status.yaml.

Gate verdict	Meaning	What you must do
PASS (≡ IR-PASS)	Plan is implementable as recorded	Sprint tracking generated; copy terminal output into readiness-gate-report.md
CONCERNS	Non-blocking gaps	Record each with mitigation + the architect's (your) sign-off; proceed only if documented
FAIL (≡ IR-FAIL)	Not implementable as recorded	Fix with the named skill (bmad-architecture, bmad-create-epics-and-stories, bmad-correct-course), re-run until PASS. **A FAIL is a compiler error — "close enough" is not accepted.**

What the audit checks (verify these yourself *before* you run it):

1. Every PRD FR/NFR trace forward to ≥ 1 story; every story traces back to a requirement (no orphans in either direction).
2. Every UX screen state (Loading / Empty / Error / Success / Denied) has a backing contract + DTO.
3. Every AD-n appears in ≥ 1 story's *Binding invariants* **and** has a red test.
4. No epic depends on a later epic; no story depends on a later story.

5. Artifacts do not contradict each other (e.g., PRD NFR says 50 ms, epic AC says 200 ms).
6. Every previous review finding (Plan-2 Phases 1–3 + this unit) is triaged and, if *Accepted*, visibly fixed.

Step 4.3 — Gate Drill (5 min, mandatory)

Deliberately **break one link** (e.g., delete the *Binding invariants* line of one story, or rename a DTO in a story but not in dto.*), re-run /bmad-sprint-planning, and record: *what the gate reported, which skill fixed it, how long it took*. Restore the link and re-run until **PASS**. This proves the gate is a real control and not a rubber stamp.

---

8. Phase 5 — Wrap-up (10 min)

1. Update ARCHITECTURE.md to **v2** (link to contracts.md, ports-and-adapters.md, schemas/).
2. Complete ai-log/decision-log-solu2.md (see requirements below).
3. Final run: /bmad-testarch-trace → test/traceability-matrix.md shows **100%** of P0 requirements linked to a story and a red test.
4. Run /bmad-walkthrough (optional) to rehearse the 3-minute explanation each partner must be able to give.
5. Tag the repository: git tag solu2-ready.

---

9. Suggested submission Package

```
├── planning/
│   ├── prd.md                      # from Plan-2 (updated only via triaged findings)
│   ├── ARCHITECTURE.md             # v2: gap fixes + links to the detailed design
│   ├── detailed-design/
│   │   ├── contracts.md            # P/Q/I x3 operations, error catalog, binding table
│   │   ├── ports-and-adapters.md   # layer map + Mermaid component diagram
│   │   └── schemas/
│   │       ├── 001_<module>_schema.sql
│   │       ├── dto.(py|ts)
│   │       ├── events.md
│   │       └── openapi.yaml
│   ├── epics.md                    # 3-4 epics, 9-16 stories, Given/When/Then, DoD
│   └── readiness-gate-report.md    # terminal output of /bmad-sprint-planning + sign-off + gate drill log
├── implementation-artifacts/
│   └── sprint-status.yaml          # generated by /bmad-sprint-planning
├── test/
│   ├── test-design.md              # risk register P0-P3, levels, thresholds
│   ├── atdd/<module>_contracts.test.(py|ts)   # executable RED suite
│   ├── fixtures/                   # fake_clock, seed_fleet, sim_telemetry, db
│   ├── nfr-evidence.md
│   ├── traceability-matrix.md
│   └── ci/quality-gate.yml (or Makefile)
├── reviews/
│   ├── review-arch-implementability.md     # Phase 0
│   ├── review-detailed-design.md           # Phase 2 (adversarial + edge-case-hunter)
│   └── review-test-quality.md              # Phase 3 (bmad-testarch-test-review)
└── ai-log/
    └── decision-log-solu2.md
```

The Plan-2 task names the backlog file epics-and-stories.md. For Solu-2 the file is **epics.md** because that is the filename bmad-sprint-planning discovers by default. If you already created epics-and-stories.md, rename it (content is the same) and update links.

AI Decision Log (Solu-2) — Requirements

- **≥ 6 significant new entries** (Plan-2 entries stay in decision-log.md), covering at least Phases 1, 2, 3, and 4.
- **≥ 2 rejected / corrected AI proposals**, with technical rationale and the final text you adopted.
- For every entry: Phase | Persona/Skill | Prompt (summary) | AI output (summary) | Human decision (ACCEPT / MODIFY / REJECT) | Reason | Artifact changed.
- **Pair attribution:** mark which partner made each decision.

10. Evaluation Rubric

Evaluation Component	Weight	Criteria for Full Marks (Exemplary Submission)
1. Contract Derivation (P, Q, I)	**20%**	3 operations with complete P/Q/I, typed error catalog, concurrency/idempotency/time clauses, **zero elastic language**, every invariant bound to an AD-n and to an enforcement point.
2. Ports & Adapters, Schemas & DTOs	**20%**	Correct hexagonal placement and domain purity (injected Clock, simulated adapters), SQL with ≥ 2 physically enforced invariants, primitive-typed DTOs, error/explanation DTOs, OpenAPI/events consistent with contracts.
3. Test Preparation (TEA)	**20%**	Risk register with P0 edge case; fixtures; **executable red-phase** suite with deep assertions (state, events, *non*-effects, explanations); CI gate; NFR evidence plan; traceability matrix with no orphan AD-n.
4. epics.md & Sprint Planning	**20%**	3–4 vertical epics, 9–16 contract-bound stories with strict Given-When-Then (happy + failure), DoD; gate drill documented; real **PASS** output; valid sprint-status.yaml.
5. AI Governance & Decision Log	**10%**	≥ 6 entries, ≥ 2 documented rejections with deep reasoning, pair attribution, clear proof of human direction.
6. Adversarial Review & Triage	**10%**	All 3 reviews executed; every finding Accept/Defer/Reject with reasoning; Accepted findings visibly fixed in the artifacts.

Automatic deductions: production logic implemented (−20 pts, stop-point violation) · tautological tests (−5 pts each, max −15) · gate verdict claimed without output (−10 pts) · cross-module dependency introduced (−10 pts).

Annex A — Seed Table: Candidate Core Operations per Module

These are **starting points**, not answers. Your own PRD and ARCHITECTURE.md take precedence; deviations must be justified in the decision log. For every module the **Mandatory Edge Case** from the Plan-2 Annex must be covered by a contract invariant, a physical enforcement, a P0 risk, a red test, and a dedicated story.

#	Module	Candidate core operations (pick 3)	Seed invariant to make physical	Typical enforcement point	Edge-case red test (must exist)
1	**User, Role & Subject Access**	resolveEffectivePermissions(userId, courseId, at) · revokeCourseAccess(userId, courseId) · ingestRoster(batch)	Permissions are resolved **per course context**; conflicting tiers are never merged by "max across courses"; revocation completes ≤ 60 s	Pure resolver in domain/; UNIQUE(user_id, course_id); revocation job + revoked_at	Student in CS101 (0 GPU) and TA in CS-480 (high priority) → each context resolves independently
2	**Academic GPU Policy Engine**	evaluateRequest(req, clock) · activateEmergencyFreeze(window) · reevaluateRunningJob(jobId, at)	Decision is a **pure function** of (request, policy_version, instant); always ALLOW, DENY or CONSTRAIN with ≥ 1 cited rule ID; deny-overrides precedence	Pure evaluator; immutable policy_versions; decisions log with rule_ids NOT NULL	Job approved at 21:58 under night policy running past 10:00 class window → re-evaluation yields DENY/terminate at boundary
3	**GPU Lab Digital Twin**	ingestHeartbeat(nodeId, payload, at) · transitionNodeState(nodeId, signal) · scheduleMaintenanceDrain(nodeId, window)	State changes only along allowed edges; **hysteresis** (N consecutive signals) before changing state; ≤ 1 alert per (node, condition) per cooldown	State-machine table in domain; node_state_history; UNIQUE(node_id, condition, cooldown_bucket)	Flapping node (on/off every 5 s) → ≤ 1 state change and ≤ 1 alert in the window

#	Module	Candidate core operations (pick 3)	Seed invariant to make physical	Typical enforcement point	Edge-case red test (must exist)
4	**GPU Reservation System**	createReservation(resources[], window, owner) · suggestAlternatives(request, k=3) · releaseNoShow(now)	No two CONFIRMED reservations overlap on the same GPU (half-open [start, end)); multi-resource request is **all-or-nothing**	Postgres exclusion constraint EXCLUDE USING gist (gpu_id WITH =, tstzrange(start,end) WITH &&) or BEGIN IMMEDIATE + overlap query	25 nodes requested, node #17 fails → 0 rows persisted, typed error, explanation lists failing node
5	**Model Catalog & Entitlement**	registerModel(meta) · resolveEntitlement(userId, modelId) · deprecateModel(modelId, replacementId)	Model visible/invocable **iff** course entitlement ∧ VRAM-tier eligibility ∧ license accepted; deprecation never orphans a replacement pointer	CHECK(min_vram_gb > 0), UNIQUE(model_id, version), entitlement join in domain	40 GB-min model + workstation-only user → denied with tier explanation; allowed for server-tier user
6	**Model Distribution Planner**	generateDistributionPlan(demand, nodes, caps) · selectEvictions(node, neededGb) · validateBandwidthBudget(plan)	After the plan, for every node used_gb ≤ capacity_gb − reserve_gb; aggregate concurrent transfer rate ≤ 1 Gbps at **every** time step; same input ⇒ same plan (determinism)	Pure planner + validate(plan) post-condition; seeded tie-breaking	No single node has enough free space → multi-node partial-eviction plan, or explainable infeasibility
7	**Local LLM Inference Gateway**	authorizeAndRoute(req) · enforceRateLimit(token, at) · circuitBreaker.record(workerId, outcome)	Auth and entitlement check **precede** any worker call; breaker follows CLOSED→OPEN→HALF_OPEN; 429 always has Retry-After	Middleware order test; breaker state machine in domain; token-bucket table/atomic counter	Token expires mid-SSE-stream → stream ends with a terminal error event; no further chunks
8	**Token Usage & Quota**	reserveTokens · commitUsage · releaseExpiredReservations *(worked pattern in §4–§6)*	committed + reserved ≤ limit always; commit idempotent per reservation	CHECK + conditional UPDATE	2 streams × 4,000 tokens with 500 left → exactly one grant of 500

#	Module	Candidate core operations (pick 3)	Seed invariant to make physical	Typical enforcement point	Edge-case red test (must exist)
9	**Night Training Scheduler**	submitJob(spec, at) · activateWindow(clock) · evictAtDeadline(at)	No student job is RUNNING outside [22:00, 06:00) beyond eviction grace; legal edges only (QUEUED_PENDING_WINDOW→RUNNING→CHECKPOINTING→EVICTED/COMPLETED)	State-machine guard; scheduler is a function of Clock; jobs.status CHECK	Checkpoint fails at 05:45 → node cordoned, user notified, GPU allocation released
10	**Distributed Training Job Planner**	planGang(request, fleet) · validateTopology(plan) · releasePartialAllocation(planId)	Gang is **all-or-nothing**; lock acquisition in canonical node-ID order **or** quorum-with-timeout so no two jobs hold partial subsets forever	UNIQUE(node_id) WHERE status='HELD'; ordered lock acquisition; timeout job	Two jobs need overlapping node sets with neither reaching quorum → one proceeds, other released; no deadlock
11	**Observability Console**	ingestMetricBatch(batch) · evaluateAlertRule(rule, window) · buildUtilizationReport(from, to)	Memory is bounded (ring buffer, fixed cap); downsampling is deterministic; alert fires once per incident until cleared	Bounded buffer in domain; alerts.incident_key UNIQUE	5,000 events/s storm → bounded memory, deterministic downsample, no UI crash, dropped-count visible
12	**AI Operations Copilot**	answerQuery(q, state) · proposeAction(intent) · verifyCitations(answer)	Every factual claim cites a state-record ID; unknown entity ⇒ ENTITY_NOT_FOUND; a destructive tool never runs without dry-run + confirmation token	Citation verifier post-condition; tool allow-list; confirmations table	Query about non-existent user → exact Entity Not Found in State, no narrative

Annex B — Readiness Checklist (before /bmad-sprint-planning)

- ☐ 3 operations have complete P, Q, I; zero banned words
- ☐ Typed error catalog covers every P-violation and every failure state
- ☐ Every AD-n has an enforcement point **and** a red test
- ☐ domain/ has no framework, DB, network, wall-clock or random imports
- ☐ Schema contains ≥ 2 physically enforced invariants; contended writes are atomic
- ☐ DTOs/OpenAPI/events consistent (no field-name drift)

- ☐ ATDD suite is executable, red for the right reason, covers the Mandatory Edge Case
- ☐ Risk register P0 items ↔ tests ↔ stories are linked in the traceability matrix
- ☐ epics.md has no forward dependencies
- ☐ Every story has Traceability, Binding invariants, Files, Schema/DTO, ATDD path, Gherkin (happy + failure), DoD
- ☐ All review findings triaged; Accepted ones fixed
- ☐ Decision log meets minimums, with pair attribution
- ☐ No bmad-build / production logic was run or written